

Technische Hochschule Ingolstadt

Fakultät Informatik

Augmented and Virtual Reality Applications
Practical Exercises

01: Introduction to Unity Hub & Unity

Introduction to AR/VR

Overview

- General Introduction to Unity
- Online Tutorial 1: Unity Hub (5-10 min.)
- Online Tutorial 2: Unity (5-10 min.) and Navigation in 3D
- Exercise 1, Part 1: GameObjects in Unity
- Exercise 1, Part 2: Colors, Textures, Materials and Maps
- Exercise 1, Part 3: Prefabs & Hierarchy
- Exercise 1, Part 4: Physics
- Exercise 1, Part 5: 3D-Models
- Exercise 1, Part 6: Tidying up Projects
- Exercise 1, Part7: Make a build / executable file
- Exercise 1, Part 8: Experimenting with Unity
- Homework: Using git/github with Unity
- Quiz

General Introduction to Unity

Introduction to Unity, Part 1

Unity is very popular in game development and has increasingly become a standard platform for the development of VR/AR applications.
It provides key features: graphics, sound, animation, and physics.

It manages so called “assets” such as images, 3D models, sounds.
Unity acts as an assembly platform for assets created with other tools.
With Unity, one can create applications for most major VR/AR devices.

This exercise introduces important Unity basics.

Step 1:
You need to create an Unity-Account.

- Register at http://unity3d.com
- This allows you to download assets (any type of virtual objects, e.g. 3D-models, audio files etc.) from the Unity Store.
- For developing C#-Scripts, you can use **Visual Studio Code.** No need for a Microsoft Account. For **Visual Studio**, you'd need one. Visual Studio Code is more than sufficient for this course.

Unity Version

- Multiple Unity versions are installed on the computers.

- We will use Unity 6.3 LTS (“Long Term Support”) for the exercises.
  The other digits are daily builds, changing constantly. As of September 2026, we’ll use Version *6000.3.11f1* (I’ll double-check the latest installs during our Oct 16th exercise and let you know.)

Important terms to know before we start our first exercise.

On the following slides, you'll learn a few important technical terms and functionalities.
Read and learn them carefully. Don't skip them. Take your time.

Important terms to know before we start our first exercise.

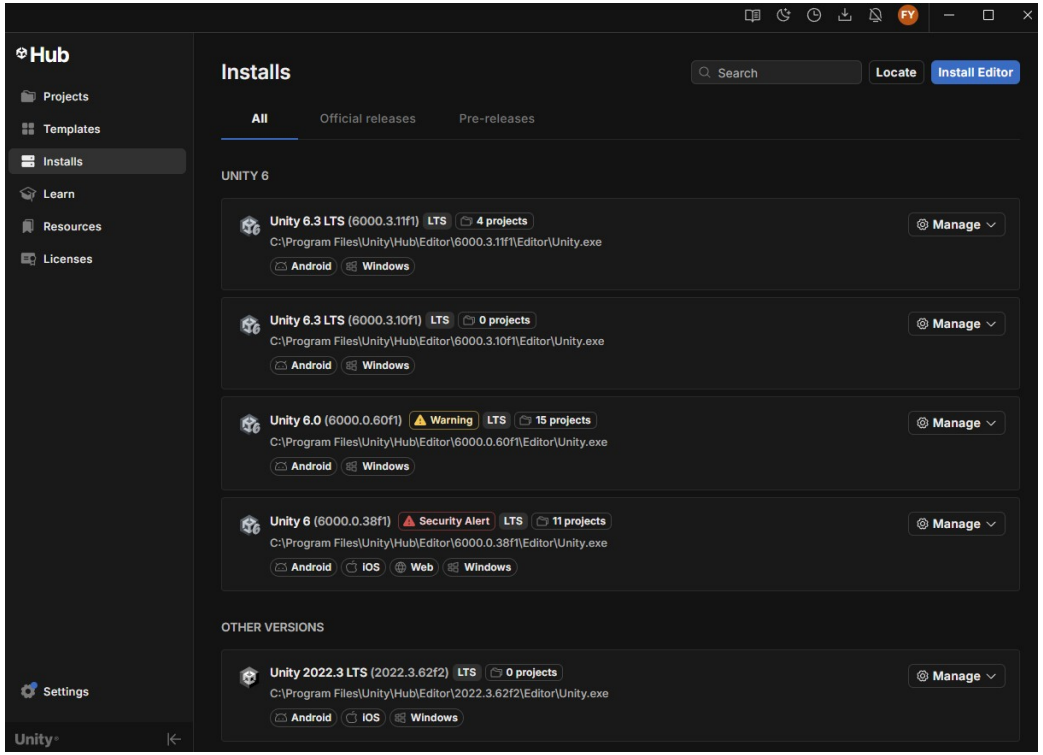
- **“Project”:** A game or application.
- **“Scene”:** A project is build of multiple “scenes” (it has at least one scene (called “SampleScene”)). Scenes contain complete virtual worlds and can represent different levels. A “scene” in Unity is different than a “scene file” we know from other software.
- **“Game Object”:** The basic building block of a scene. **Every object in a scene is a GameObject**: A “Camera” is a GameObject. A 3D-“Sphere” is a GameObject. So is a “Light”.
- **“Component”**: Gives a GameObject various properties and abilities.
  You can attach as many components as you want. A component can be a Script, it can be a Material, a Transform, etc.
  Components make up a GameObject become a distinct Object: A Camera has other Components than a Cube.

Important terms to know before we start our first exercise

- **“Asset”**: Digital content belonging to a project (e.g., images, models, scripts, textures, sounds, clips).
- **“Prefab”**: Prefabs are reusable, predefined GameObject templates. Think of a prefab as a blueprint for GameObjects. Editing a prefab updates all its instances. More efficient than creating GameObjects independently.
- **“Collider”**: An invisible bounding shape that enables physical collisions of a GameObject.

Unity Hub
The Unity Hub is the central tool for managing Unity. It gives access to:

- Projects
- Templates
- Versions (“Installs”)
- Tutorials
- Resources
- Licenses

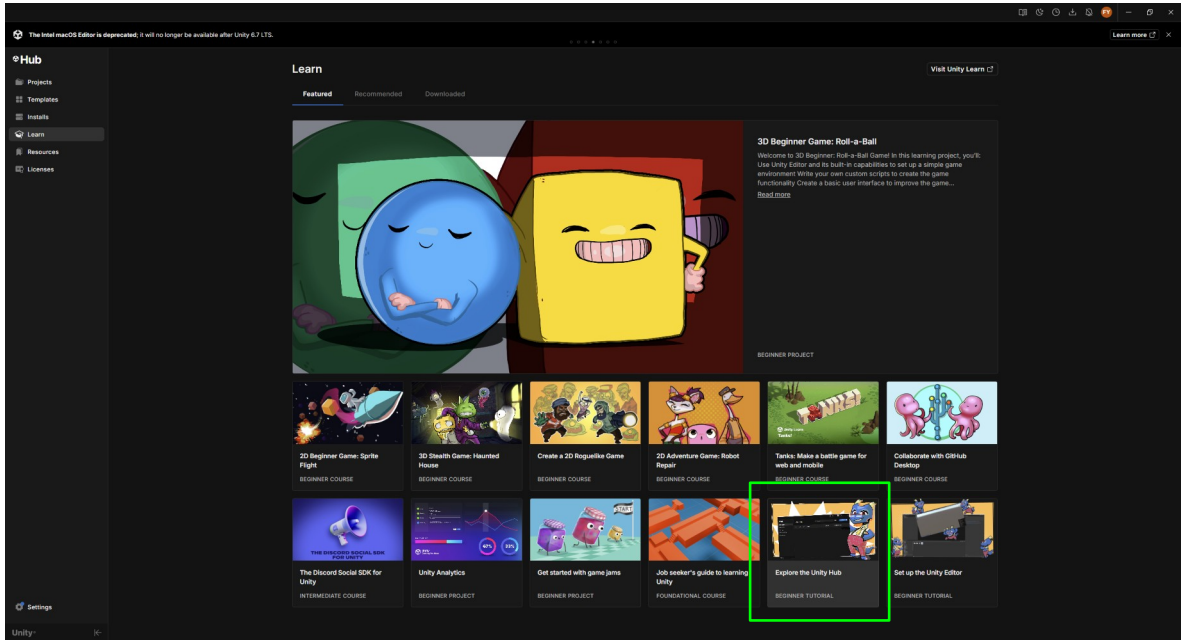


Tutorial 1: Unity Hub (5-10 min.)

Unity Hub
Before we dive into Unity, we'll start with an introduction tutorial into the Unity Hub (~ 10 minutes max.) and Unity itself (~10 min.):

Launch Unity Hub. Left side bar: "Learn"

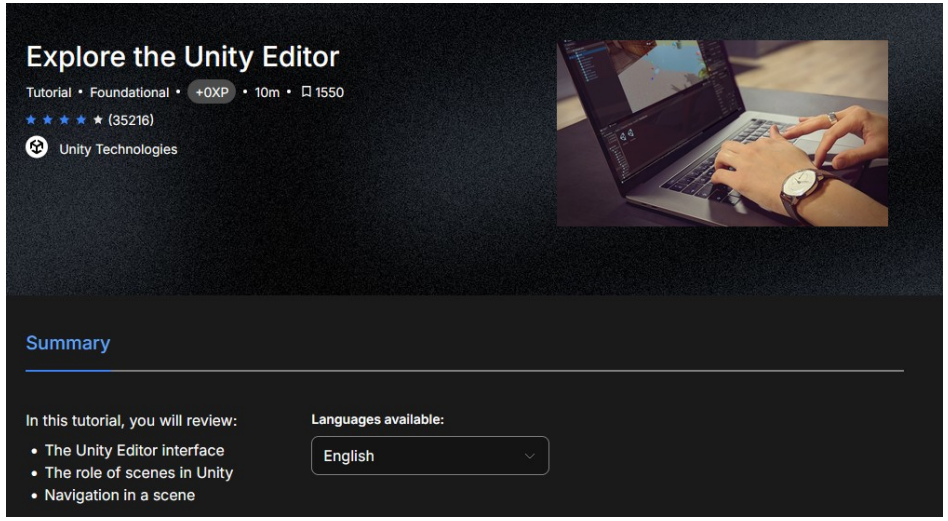
- 1. Launch "Explore the Unity Hub"
- 2. Lower right corner → "View on Unity Learn" (In **Steps 2 & 3: do not install. Read and understand.**)



Tutorial 2: Unity Editor (5-10 min.)

Tutorial 2: “Explore the Unity Editor”

- *https://learn.unity.com/tutorial/explore-the-unity-editor-1*
- Get familiar with navigation, shortcuts.
  Try things out.
- Once completed, go to next slide.



Exercise 1: First Project

Exercise 1 (First Project)

a) In the project you created in the previous tutorial: Delete the Cube (or any GameObjects created so far). The empty default scene should only contain the Main Camera and Directional Light.

b) Select the camera in the Hierarchy Pane or Scene View. Its components are shown in the Inspector Pane. Like every GameObject, it has a *Transform component* defining its position, rotation, and scale relative to its parent (Local Coordinates) or the world (Global Coordinates).
For the camera, a rotation of (0, 0, 0) means it faces the *positive z-axis*.

The *Camera component* turns the Main Camera GameObject into a camera and allows you to set properties such as projection, field of view, viewing distance, and background.

The Directional Light illuminates the scene with parallel rays, simulating a distant light source such as the sun.

c) Next, we'll add the first additional predefined object to the scene via the menu:

- GameObject → 3D Object → Sphere. Briefly explore the available options.
- Rename the sphere directly in the Hierarchy or Inspector Pane. In complex scenes, GameObjects should have meaningful names, similar to variables in programming.
- The Inspector Pane shows the sphere's components and properties. Besides the Transform Component, it has a Mesh Filter and Mesh Renderer:
  The Mesh Filter component tells a GameObject which 3D mesh (shape/geometry) it should use. A mesh is a 3D geometry object made up of Vertices (Points), Edges and Polygons.
  The Mesh Renderer controls the sphere's visual representation.
- The sphere is stored as a polygonal mesh consisting of Vertices (Points), Edges, and Polygons.
- The Sphere Collider enables collision detection with other objects.

Exercise 1

d) Make sure the sphere is located at the origin:

- In the Inspector Pane, check if the position is (0, 0, 0). If not, select “Reset” from the small 3-points-menu in the upper right of the “Transform” component.

- Transformation:
  It is recommended to always check the transformations of objects when creating them.
  Transformations are always applied dependent on parent objects. Try to understand what this sentence means not only in theory. Think of an example and how it affects Game Objects parented to others in a hierarchy.

- Get used to saving your scene regularly: Ctrl+S (or on german keyboard: STRG+S).

Colors and Textures

a) The visual appearance of an object is largely determined by the material.
Among other things, it determines color (“Base Map”), reflection, glossiness, roughness, and emission.
Currently, the sphere has a non-editable default (and greyed out) “Lit (Material)” component assigned.

- Create a new material: Menu → Assets → Create → Material. Or, in the Assets window view: RMB → Create…
- Give it a meaningful name right away. For this new material, we can select a new color in the Base Map. Easy to find out how.
- Now you can drag & drop the new material from the Assets View onto the sphere - either in the Hierarchy Pane or directly in the Scene View.
- Many things in Unity work with drag & drop.

Colors and Textures

b) Another material property often used is a *texture*. **A *texture* is an image applied to a 3D object.**

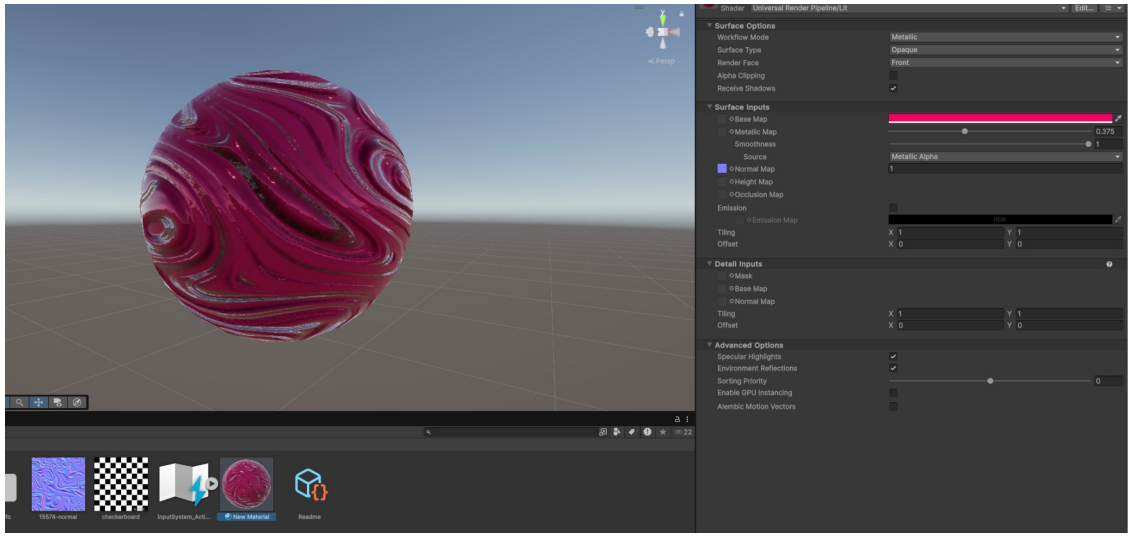
- Download any .gif, .png, or .jpg image. Or find one in your file explorer.
- Import it into the Assets View: Drag & drop from File Explorer. Or RMB → Import New Asset.
- Dragging the texture directly onto the sphere replaces the Base Color.
  But we don't want this. Revert this step: instead of Ctrl-Z, select the sphere in the hierarchy, check the Inspector Pane → Material → Element 0, and remove your texture.
- Now, select the Material and drag the texture into the square field next to Base Map.
  The texture is now **blended** with the existing Base Color, providing more control.

Colors and Textures

c) Other types of Maps we will adjust: *Metallic Map* and *Normal Map*.

- Find a “normal map” online and download the image. Normal maps usually have similar colors: mainly blue-ish/purple with some red/pink & green in it. Try to understand why. Ask.

- Drag the normal map from the Assets View into the small field next to “Normal Map” in the Material Inspector.The sphere should now appear to have surface depth and detail. Try to understand why and how.



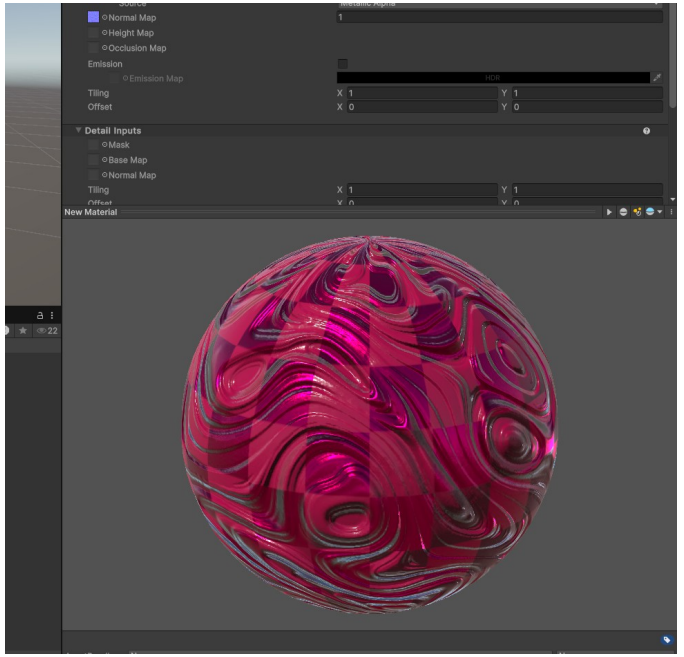
Practical Exercises

Exercise 1 – Part 2

Colors and Textures

- You will also find a “Metallic Map” field. Download any high-contrast black-and-white image. Drop it into Assets View.

- Assign the same texture as the Metallic Map. Inspect the Material preview in the lower-right corner of the Inspector. Observe how different areas of the texture affect the sphere’s metallic appearance: Where does the sphere appear most metallic? Try to understand.



Prefabs and Hierarchy

A Prefab is a reusable template or blueprint for a GameObject. Multiple instances can be created from the same Prefab, and changes to the Prefab can be applied to all its instances.

a) Create a Prefab:
Drag the sphere from the Hierarchy into the Assets View. The sphere is now a Prefab. Delete the original sphere from the Hierarchy.

b) Use the Prefab:
Create an empty GameObject: GameObject → Create Empty (Ctrl Shift N). Rename it to *3D-Cross*.
The GameObject already has a Transform component, which allows it to be positioned and moved. Set its position to (0, 0, 0).
Tip: The Transform component has a “Reset” option that does this with a single click. Try to find it.

Prefabs and Hierarchy

c) Note that prefabs are written in blue letters in the Hierarchy Pane and have a and blue sphere icon.

- From the Assets View, drag the sphere prefab onto “3D-Cross” in the Hierarchy.
- Rename the sphere to “Center” and place it at the origin (0, 0, 0).
- Repeat the same process six more times with the following names and positions:
- Left (–2, 0, 0), Right (2, 0, 0), Up (0, 2, 0), Down (0, –2, 0), Front (0, 0, –2), Back (0, 0, 2)
- If necessary, re-orient the camera to see all spheres. Attention: Even if you navigate in the viewport, the camera stays where it is. To bring the camera into the current view: select the camera in the Hierarchy: Menu → GameObject → Align With View (Or: Ctrl Shift f)
- Now, all seven spheres are hierarchically parented to “3D-Cross”; their positions are relative to it. Means: to move all spheres, we can simply move “3D-Cross”. This is a classical Parent→Child relationship in all 3D applications.

Practical Exercises

Exercise 1 – Part 4

Physics

Now we'll add some physics and gravity to our objects.

a) Select the sphere prefab in the Assets View.

- In the Inspector Pane, click "Add Component" and add the component "RigidBody" to the prefab (use the search field). The prefab now becomes a rigid body (a physical object) with some mass which responds to gravity and can detect collisions.
- The changes automatically apply to all prefab instances.

b)

- Next, add a Plane (GameObject → 3D Object → Plane)
- Place it below spheres: (0, –5, 0). Scale uniformly to 5× on all axes. Find out how to lock/link X, Y, Z scale so you only enter the value once.

Physics

- By default, a plane is 10 by 10 meters. With the scaling we just applied, we expand it to 50 m × 50 m.

c)

- If you now enter the “Play Mode” by pressing the Play button above the Scene View Pane, all the spheres fall downwards and remain lying on the plane.
- If necessary, re-adjust the camera so that the simulation can clearly be seen.
  Now exit the the play mode (Stop button).
- Note: If you make changes in the Inspector Pane during Play Mode (= runtime) and then exit Play Mode, the changes are lost. Only edits made in Scene Editor can be saved.
- Save the project.

Practical Exercises

Exercise 1 – Part 5

Importing a 3D-Model
Unity is not suitable for modeling (the creation of 3D models). This is done in other 3D-Applications (e.g. Houdini, Blender, or CAD Software).

- There are multiple 3D-model-formats: .obj, .fbx, .usd, .gltf, .abc and many more. We'll stick to .fbx for now.
- Download any **free** 3D model in fbx-format from *polyhaven.com*, *free3d.com*, or *turbosquid.com*.
- Drag it into the Unity Assets Pane/View. Drag & drop into Scene View Pane.
- A model imported this way is always a prefab.
- (Optional:
  If you are familiar with a modeling software (e.g. Blender), you can import the FBX model in there and, for example, select and move individual points of the geomtery in Edit View. After saving, the changes should be immediately visible in Unity.)

Practical Exercises

Exercise 1 – Part 6

Tidying up our Project
Now we'll clean up our project.

- It has proven useful to set up the following subfolder structure in the Assets View Pane (Right click → Create → Folder) and move with the corresponding assets into these subfolders:
- Scenes
- Textures
- Materials
- Prefabs
- Sounds (we don't have any in this project)
- Scripts (not in this project)

Creating an Executable File

We can now make a build from our project, i.e., create an executable file from the Unity application.

- Menu → File → Build Profiles (Ctrl Shift b)
- "Scene List" shows available scenes in the project. Unless you did not create a new scene, "SampleScene" should be the only and correct one.
  If it doens't show up in the list: Click "Add Open Scenes", and add the currently opened scenes to the build.
- Select the correct target platform: Windows (should be active)
- After clicking "Build", you will be asked to choose a target directory and specify a name for the executable file:
  We create a subfolder called "Builds" in the project directory and save the file in there.
  Once the build process has finished, you'll find the executable file with the chosen name in this subfolder.
- Start it by double-clicking the newly created .exe file to test it.
  To run the application on another computer, in addition to the executable file you must also transfer the subfolder with the suffix "{YourProjectName}_**Data**" inside the "Builds" folder (was created during the build).

Experimenting, play with Unity

- If you are done so far: play around. Experiment. Try.
  Create a 3D world with textured objects and imported 3D models.
- Being registered now, you can download a large number of models and pre-made GameObjects from the Unity Asset Store, for example trees, rocks, etc:
  Menu → Window → Asset Store (another place to look for and download models)
- (Therefore, registration is recommended).

Homework

Practical Exercises

Homework

Using Unity with github
Before next Friday's practical exercise, learn the basics of using *GitHub* with Unity, using either:

- The *GitHub Desktop App (GUI),* Tutorial: *https://youtu.be/tTIlDb71t2s?t=74* (starts at 1:14 intentionally).
- Or the command line (typical commands are ***add, commit, push***)
- Either way: download the "unity.gitignore" file either from *here* or from *Moodle.*
  Place it in the Unity project directory, at the same level as the "*Assets*" folder.
  If a GitHub repository contains multiple Unity projects, each project needs its own .gitignore file.
- Why use .gitignore? Unity generates many temporary and runtime-generated files that are not needed when transferring a project. The .gitignore file prevents these unnecessary files from being pushed to or pulled from GitHub.
- This helps avoid Gigabytes of unnecessary data transfer, thousands of redundant files, and hence larger, slower GitHub repositories. In short:
  .gitignore ensures that GitHub tracks and transfers **only the files needed for your Unity project**.

Exercise Questions (Exam-Relevant)

Exercise Questions

1.) Explain the following important Unity Game Engine terms (short sentence(s)):
Project, Scene, GameObject, Component, Asset, Prefab, Collider

2.) What functionalities do the following sub-windows (“Panes”) in the Unity Editor have:
Hierarchy, Scene View, Inspector Pane, Project View, Console

3.) For a camera object in Unity, in the Inspector Pane, tab Camera → Projection, you can set the two values “Near” and “Far” Clipping Planes. What do these values mean? What is their purpose?

4.) Which component does every GameObject in Unity have? What can you change with this component?

5.) Indicate how the three coordinate axes in Unity are oriented when viewed from the camera
(answer format: name of axis; values increase from … to …).
Also, state whether Unity’s coordinate system is left-handed or right-handed.

6.) What are empty GameObjects in Unity good for?